

# MySQL Server Architecture Deep-Dive

wilhasse/courses

## MySQL Server Architecture Deep-Dive

[Start here](#)

[Chapters](#)

[How to study](#)

### Chapter 0 — Overview: The Journey of a SQL Statement

[Two halves of one database](#)

[The journey \(memorize this picture\)](#)

[Chapter roadmap](#)

[How this connects to the InnoDB series](#)

[How to study with this series](#)

[Conventions](#)

### Chapter 1 — Startup, Connections & Command Dispatch

[1.1 Boot: `mysqld\_main\(\)`](#)

[1.2 From `accept\(\)` to a session thread](#)

[1.3 THD: the session object](#)

[1.4 The command loop and the two dispatch levels](#)

[1.5 What to remember](#)

### Chapter 2 — The Parser

[2.1 Three representations, two steps](#)

[2.2 The lexer](#)

[2.3 The grammar and the parse tree](#)

[2.4 The output structures: LEX, Query\\_block, Item](#)

[2.5 What to remember](#)

### Chapter 3 — Resolution & Prepare

[3.1 Where we are](#)

[3.2 Opening tables: shares, instances, references](#)

[3.3 `Query\_block::prepare\(\)`: name resolution](#)

[3.4 Permanent transformations](#)

[3.5 Prepared statements: why this phase is separable](#)

[3.6 What to remember](#)

### Chapter 4 — The Query Optimizer

[4.1 The pipeline: `JOIN::optimize\(\)`](#)

[4.2 Costs and statistics](#)

[4.3 The two searches: range analysis and join ordering](#)

4.4 The output: an AccessPath tree

4.5 What to remember

## **Chapter 5 — The Iterator Executor**

5.1 The execution loop is ten lines

5.2 The RowIterator contract and the zoo

5.3 filesort: ORDER BY without an index

5.4 Internal temporary tables

5.5 What to remember

## **Chapter 6 — The Handler API: Where SQL Meets the Storage Engine**

6.1 Two objects define an engine

6.2 Crossing the bridge into InnoDB

6.3 The impedance mismatches

6.4 What to remember

## **Chapter 7 — Metadata Locks & Transaction Coordination**

7.1 Why the server needs its own locks

7.2 The intention lock on COMMIT

7.3 The distributed transaction inside every MySQL server

7.4 What to remember

## **Chapter 8 — The Binary Log**

8.1 Why a second log?

8.2 Events on the wire

8.3 Group commit: the three-stage pipeline

8.4 GTIDs: names for transactions

8.5 What to remember

## **Chapter 9 — Replication**

9.1 The pipeline: three threads and two logs

9.2 Parallel apply (MTS)

9.3 Crash safety and semi-sync

9.4 What to remember

## **Chapter 10 — The Data Dictionary & Atomic DDL**

10.1 The 8.0 revolution: no more .frm files

10.2 The dd:: object model

10.3 Atomic DDL: the two-dictionary problem, solved properly

10.4 ALTER TABLE: COPY, INPLACE, INSTANT

10.5 What to remember

## **Chapter 11 — What Percona Adds**

11.1 Thread pool (a Connection\_handler)

11.2 MyRocks (a handler)

11.3 Security & operations plugins/components

11.4 Observability patches

11.5 What to remember

## **Chapter 12 — Capstone: One Statement, Every Layer**

12.1 The statement

12.2 The full trace

12.3 The five ideas you now own

12.4 Where to go next

# MySQL Server Architecture Deep-Dive

---

A 13-chapter, step-by-step study guide to the MySQL **server layer** — everything above the storage engine — based on the source of **Percona Server 8.0.46** (MySQL 8.0 LTS + Percona enhancements). It follows the journey of a SQL statement top-down: socket → parser → optimizer → executor → handler API → binlog → replica, with `file.c:line` references and Mermaid diagrams throughout.

**Companion series:** [InnoDB Architecture Deep-Dive](#) covers everything *below* the handler API. The two series meet at Chapter 6 here — read either first, but together they cover the whole database.

**Source studied:** [percona/percona-server](#) (8.0 branch). File references like `sql/sql_parse.cc:1361` point into that tree; line numbers are anchors from 8.0.46 and may drift slightly across releases.

## Start here

➔ **Chapter 0 — Overview: The Journey of a SQL Statement**

📖 Also available as a [website](#) and a [PDF](#) for offline reading.

## Chapters

| #  | Chapter                         | Question it answers                                                   |
|----|---------------------------------|-----------------------------------------------------------------------|
| 00 | Overview                        | How do the layers fit together?                                       |
| 01 | Startup, Connections & Dispatch | How does a packet become a running command in a THD?                  |
| 02 | The Parser                      | How does SQL text become a parse tree and LEX?                        |
| 03 | Resolution & Prepare            | How are names bound, tables opened, queries transformed?              |
| 04 | The Optimizer                   | How is the cheapest plan found?                                       |
| 05 | The Iterator Executor           | How does the plan actually run?                                       |
| 06 | The Handler API                 | How does the server talk to InnoDB — or any engine?                   |
| 07 | MDL & Transaction Coordination  | What serializes DDL vs DML? How do binlog + engine commit atomically? |
| 08 | The Binary Log                  | How are changes recorded, grouped, GTID-named?                        |
| 09 | Replication                     | How do replicas stay in sync — and in parallel?                       |
| 10 | Data Dictionary & Atomic DDL    | Where does the schema live since 8.0?                                 |
| 11 | What Percona Adds               | Thread pool, MyRocks, audit, backup locks...                          |
| 12 | Capstone                        | One UPDATE traced through every layer of both series                  |

## How to study

1. Clone percona-server (8.0 branch) and keep it open beside each chapter.
2. Build with debug symbols; each chapter's **Try it** exercise uses gdb breakpoints, `EXPLAIN FORMAT=TREE/ANALYZE`, `optimizer_trace`, `performance_schema`, or `mysqlbinlog` to observe the mechanism live.
3. Finish with the capstone (Chapter 12): if you can narrate its trace from memory, you understand MySQL.

# Chapter 0 — Overview: The Journey of a SQL Statement

---

**Series:** MySQL Server Architecture Deep-Dive — a step-by-step study guide to everything *above* the storage engine, based on the source of **Percona Server 8.0** (MySQL 8.0 LTS + Percona enhancements). Companion to the [InnoDB Architecture Deep-Dive](#), which covers everything *below*.

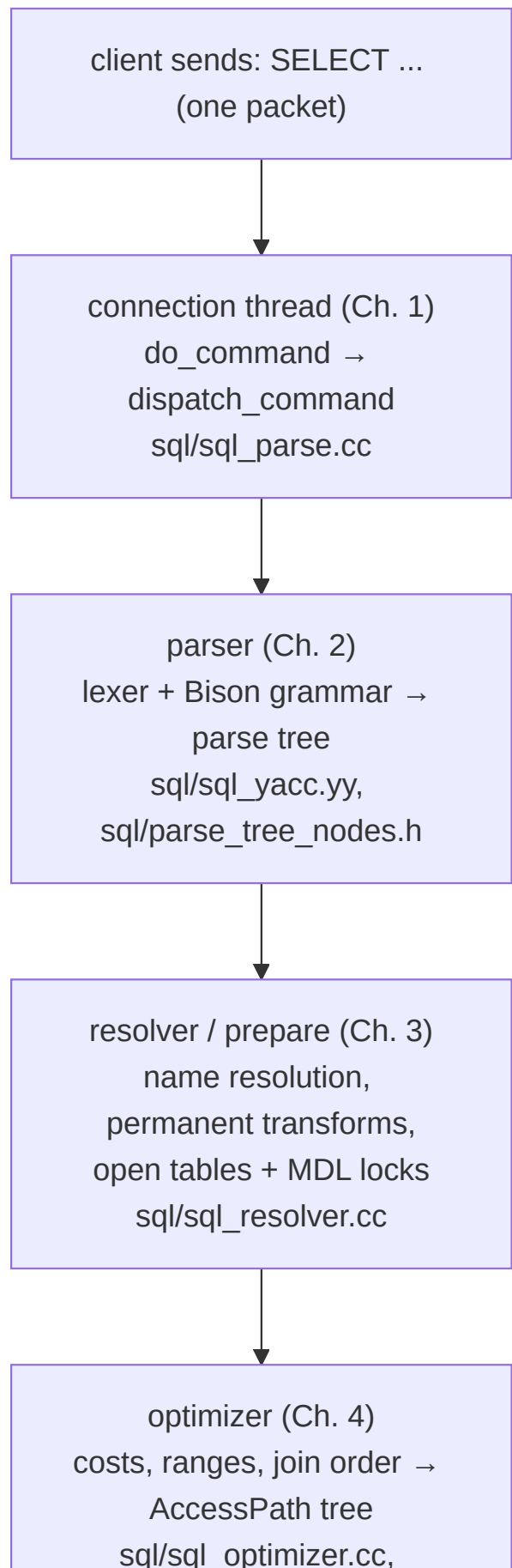
## Two halves of one database

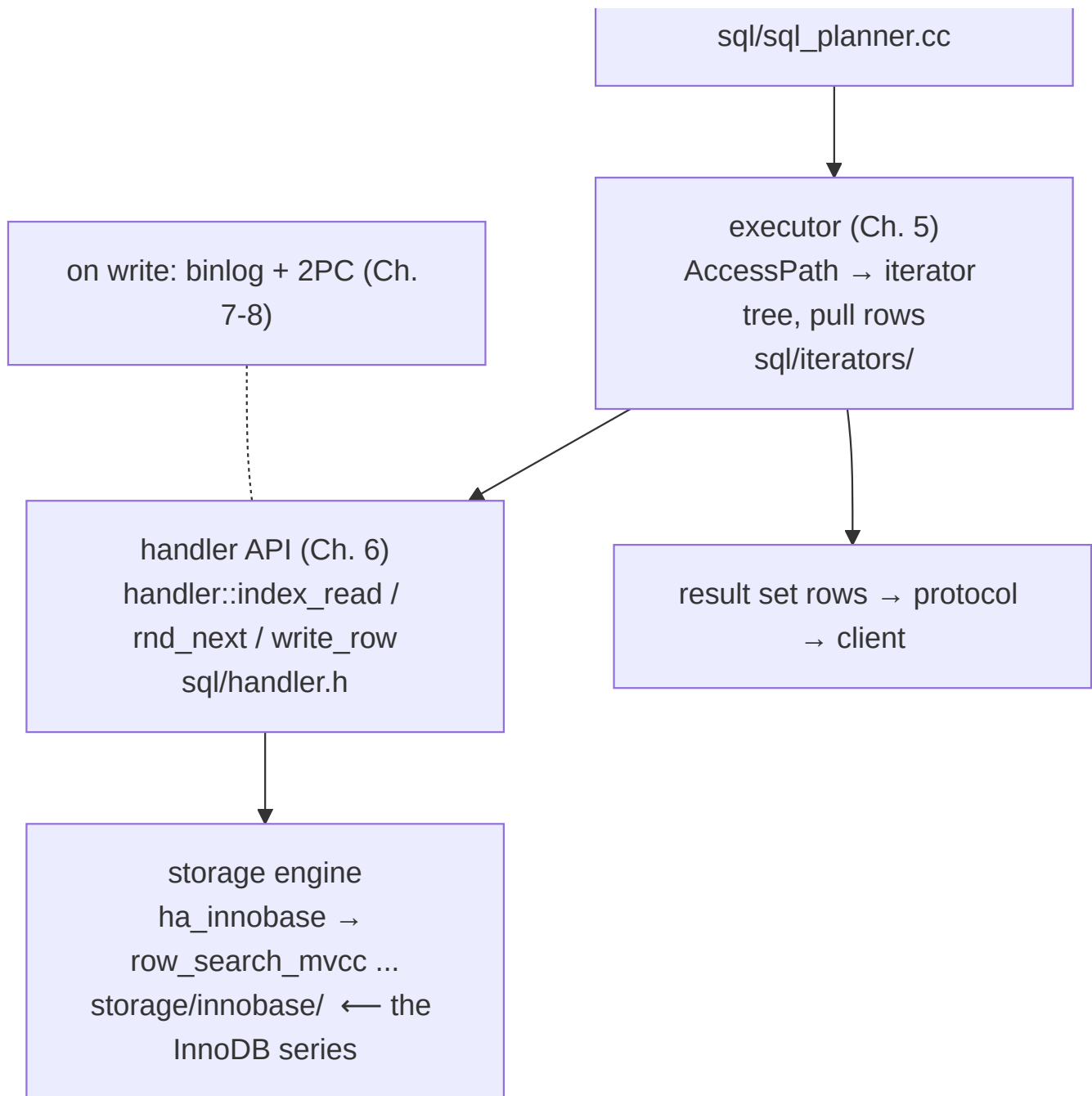
MySQL is really two programs stapled together:

- **The server layer** (`sql/`) — networking, SQL parsing, optimization, execution, replication, and everything a "database server" implies. It knows nothing about pages, B+trees, or buffer pools.
- **Storage engines** (`storage/`) — InnoDB, MyRocks, TempTable, MyISAM... Each knows how to store and retrieve *rows in tables*, and nothing about SQL.

Between them sits one C++ interface: the **handler API** (`sql/handler.h`). That boundary is this series' destination — Chapter 6 lands exactly where the InnoDB series begins.

**The journey (memorize this picture)**





Around that synchronous path run the supporting systems: metadata locking and the binlog/engine two-phase commit (Chapters 7-8), replication (Chapter 9), the transactional data dictionary (Chapter 10), and Percona's additions (Chapter 11). Chapter 12 replays the whole journey as a capstone.



## Chapter roadmap

| #  | Chapter                                   | Question it answers                                                      | Main sources                                                   |
|----|-------------------------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------|
| 01 | Startup, Connections & Command Dispatch   | How does a TCP packet become a running SQL command in a THD?             | sql/mysqld.cc, sql/conn_handler/, sql/sql_parse.cc             |
| 02 | The Parser                                | How does SQL text become a parse tree and a LEX?                         | sql/sql_yacc.yy, sql/parse_tree_nodes.h                        |
| 03 | Resolution & Prepare                      | How are names bound, tables opened, queries transformed?                 | sql/sql_resolver.cc, sql/table.h                               |
| 04 | The Optimizer                             | How is the cheapest plan found?                                          | sql/sql_optimizer.cc, sql/sql_planner.cc, sql/range_optimizer/ |
| 05 | The Iterator Executor                     | How does the plan actually run?                                          | sql/iterators/, sql/filesort.cc                                |
| 06 | The Handler API                           | How does the server talk to InnoDB (and any engine)?                     | sql/handler.h, storage/innobase/handler/ha_innodb.cc           |
| 07 | Metadata Locks & Transaction Coordination | What serializes DDL vs DML? How do binlog and engines commit atomically? | sql/mdl.cc, sql/handler.cc                                     |
| 08 | The Binary Log                            | How are changes recorded, grouped, and made crash-safe?                  | sql/binlog.cc, libbinlogevents/                                |
| 09 | Replication                               | How do replicas stay in sync?                                            | sql/rpl_*.cc                                                   |
| 10 | The Data Dictionary & Atomic DDL          | Where does the schema live since 8.0?                                    | sql/dd/                                                        |
| 11 | What Percona Adds                         | Thread pool, MyRocks, auditing, encryption...                            | plugin/, storage/rocksdb/                                      |
| 12 | Capstone: One Statement, Every Layer      | Can you narrate the whole journey?                                       | everything                                                     |

## How this connects to the InnoDB series

The **InnoDB deep-dive** studied the 2005-era embedded engine, where *you* were the SQL layer, calling `ib_cursor_*` directly. In this series the same engine (20 years more evolved) sits behind `ha_innodb`, and the "user" of its cursors is the iterator executor. Watch for the reunions:

- `row_prebuilt_t` — the prebuilt struct from the embedded API is exactly what `ha_innodb` keeps per open table.
- `row_search_mvcc` — the descendant of `row_search_for_client` (InnoDB series, Ch. 9).

- `TRX_PREPARED` and XA — the InnoDB commit machinery (Ch. 7 there) is one participant in the server's binlog two-phase commit (Ch. 7-8 here).
- The data dictionary — InnoDB's `SYS_TABLES` bootstrap (Ch. 10 there) grew into MySQL 8.0's entire transactional dictionary (Ch. 10 here).

## How to study with this series

1. **Keep the source open.** File references are relative to the `percona-server` tree (8.0 branch). The code is heavily Doxygen-commented; `sql/sql_parse.cc` and `sql/handler.h` have excellent header comments.
2. **Use a debug build + gdb.** Break on `dispatch_command`, `JOIN::optimize`, `ha_innodb::index_read` and step a simple `SELECT` through all layers.
3. **Use the server's own X-ray tools:** `EXPLAIN FORMAT=TREE` (the iterator tree of Ch. 5), `performance_schema` stage/statement tables, `SHOW ENGINE INNODB STATUS`, `mysqlbinlog --hexdump` (Ch. 8).
4. **Read `mysql-test/` cases** for any feature you study — they are executable specifications.

## Conventions

- File references: `sql/sql_parse.cc:1610` style, relative to the repo root; verified against Percona Server 8.0.46. Line numbers drift between releases — treat them as anchors, not gospel.
- Diagrams are Mermaid (GitHub renders them) or ASCII tables.
- "8.0 change" boxes call out what differs from the 5.x architecture, since much online material describes the old world.

---

**Next:** [Chapter 1 — Startup, Connections & Command Dispatch](#)

# Chapter 1 — Startup, Connections & Command Dispatch

How a mysqld process boots, how a TCP connection becomes a session, and how a packet becomes a running SQL command. Source: `sql/mysqld.cc`, `sql/conn_handler/`, `sql/sql_class.h`, `sql/sql_parse.cc`, `sql/protocol_classic.cc`

## 1.1 Boot: `mysqld_main()`

`sql/mysqld.cc:7693` is the real `main`. The sequence matters because it encodes the dependency order of the whole server (compare the InnoDB series Ch. 11 — same idea, bigger machine):

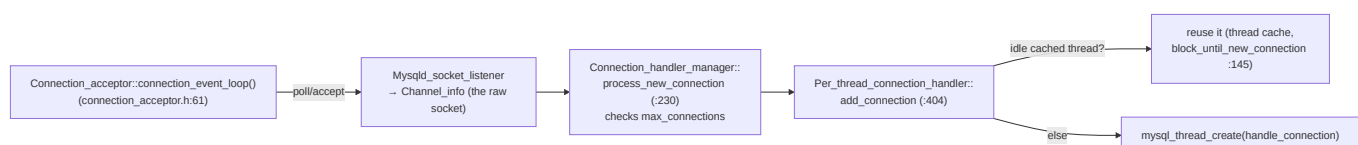
```
init_common_variables()      (:8108)  options, charsets, sysvars
init_server_components()     (:8312)  ↳ the heavy lifter:
  ↳ plugin_register_builtin_and_init_core_se (:6849)  ← InnoDB starts HERE
    (InnoDB runs its own recovery – InnoDB series Ch. 5 – before the server proceeds)
  ↳ dd::init(...)             (:6868)  data dictionary on top of InnoDB (Ch. 10)
  ↳ plugin_register_dynamic_and_init_all (:6966)  other plugins (incl. thread_pool)
  ↳ ha_post_recover()         (:7300)  XA recovery finalization (Ch. 7)
network_init()               (:8484)  listeners created
acl_init() / grant_init()    (:8552)  privileges (stored in tables → needed DD)
Events, replica threads, signal handler ...
mysqld_socket_acceptor->connection_event_loop() (:8738) ← blocks forever, accepting
```

Note the inversion from 5.x folklore: **the storage engine and dictionary come up before the privilege system**, because since 8.0 the grant tables are InnoDB tables read through the DD.

A running mysqld has surprisingly few *server* threads: the accept loop, a signal-handler thread, one thread per connection (or a thread pool — below), the event scheduler, replica threads, and `handle_manager` for periodic maintenance. Everything else (page cleaners, purge, I/O) belongs to InnoDB internally.

## 1.2 From `accept()` to a session thread

The connection path is a small class dance in `sql/conn_handler/`:



`handle_connection()` (`sql/conn_handler/connection_handler_per_thread.cc:247`) is a connection's whole life:

```
init_new_thd(channel_info);           // build the THD
thd_manager->add_thd(thd);
thd_prepare_connection(thd);         // handshake + authentication
while (thd_connection_alive(thd))
    if (do_command(thd)) break;      // ← the command loop (1.4)
end_connection(thd); close_connection(thd);
thd->release_resources(); thd_manager->remove_thd(thd);
block_until_new_connection();        // park in the thread cache for reuse
```

This is the famous **one-thread-per-connection** model — simple, low-latency, and the reason `max_connections` × per-thread memory is a capacity-planning formula. The thread cache (`thread_cache_size`) only saves thread *creation*, not memory or scheduler pressure.

**Percona's thread pool** (`sql/threadpool_unix.cc` — built into the server, activated as the `thread_pool` plugin) replaces this scheduler: connections are sharded into **thread groups**, each with an epoll listener; a bounded set of worker threads (`worker_main`, `threadpool_unix.cc:194`) picks up ready connections and runs `threadpool_process_request()` (`threadpool_common.cc:206`). Ten thousand mostly-idle connections then cost ~dozens of threads instead of ten thousand. (Oracle sells this as an enterprise feature; Percona ships it in the open build — a good example of what "Percona Server" means.)

## 1.3 THD: the session object

`class THD` (`sql/sql_class.h:1105`) is MySQL's equivalent of "everything about one session", and you will see a `THD *thd` first parameter on nearly every function in `sql/`. What it owns:

| member                                                                 | role                                           | chapter |
|------------------------------------------------------------------------|------------------------------------------------|---------|
| <code>net</code> , <code>m_protocol</code>                             | the wire connection & protocol codec           | 1       |
| <code>m_main_security_ctx</code>                                       | authenticated user, privileges                 | —       |
| <code>lex</code> , <code>m_query_string</code> , <code>mem_root</code> | current statement's parse state & memory arena | 2-3     |
| <code>open_tables</code> , <code>mdl_context</code>                    | opened table handles + metadata locks          | 3, 7    |
| <code>m_transaction</code> ( <code>Transaction_ctx</code> )            | per-engine transaction registrations           | 7       |
| <code>variables</code> , <code>status_var</code>                       | session sysvars & counters                     | —       |
| <code>killed</code> (atomic)                                           | cooperative KILL flag, polled at safe points   | —       |
| <code>binlog_cache</code> (via <code>transaction ctx</code> )          | buffered binlog events                         | 8       |

Two design points worth internalizing: **memory is arena-based** (`MEM_ROOT` — allocations live until statement/connection end, freed in one shot; the same philosophy as InnoDB's `mem_heap_t`), and **KILL is cooperative** — `thd->killed` is checked at loop boundaries, which is why a runaway query inside a storage engine can take a while to die.

## 1.4 The command loop and the two dispatch levels

Every client interaction is a **command packet**; SQL is just the payload of `COM_QUERY`. The loop (`sql/sql_parse.cc`):

```
do_command (:1361)                read one packet (blocking)
└─ dispatch_command (:1765)        switch (command) ← protocol-level verbs
    COM_QUERY (:2104)              plain SQL text
    COM_STMT_PREPARE/EXECUTE (:2066/:2025) prepared statements (binary)
    COM_PING, COM_INIT_DB, COM_BINLOG_DUMP_GTID (replication! Ch. 9), ...
    └─ dispatch_sql_command (:5514) for COM_QUERY:
        parse_sql (:5549)           → Chapter 2
        mysql_execute_command (:3110) the giant switch(lex->sql_command)
            SQLCOM_SELECT, SQLCOM_INSERT, ... → Sql_cmd::execute → Chapters 3-6
```

So there are **two switches**: protocol commands (`COM_*`, what the wire says) and SQL commands (`SQLCOM_*`, what the parser understood). Replication clients enter the same loop — a replica is just a client that sends `COM_BINLOG_DUMP_GTID` and never leaves.

### The wire protocol in one paragraph

`sql/protocol_classic.cc` implements the MySQL client/server protocol: length-prefixed packets; results as *metadata packets, then row packets* (length-encoded strings — `net_store_data`, `:1266`), terminated by an OK/EOF packet (`net_send_ok` `:860`, `net_send_eof` `:1052`); errors as ERR packets (`net_send_error` `:613`). Rows stream to the client *as the executor produces them* — there is no server-side result buffering by default, which is why a slow-reading client can hold locks (and why `SQL_BUFFER_RESULT` exists).

## 1.5 What to remember

1. Startup order encodes architecture: engines → data dictionary → privileges → network. InnoDB recovery has already finished before MySQL accepts a single connection.
2. A connection = a `Channel_info` → a `THD` → a thread running `do_command` in a loop. One-thread-per-connection is the default; Percona's thread-pool scheduler swaps in a bounded worker model.
3. `THD` is the session god-object: protocol, privileges, parse state, transaction context, MDL, kill flag. Arena allocation and cooperative kill are its two big idioms.
4. Two dispatch layers: `COM_*` (protocol) then `SQLCOM_*` (SQL). Everything you'll study in Chapters 2-6 happens inside one iteration of `do_command`.

**Try it:** `gdb -p $(pidof mysqld)`, `break dispatch_command`, run `SELECT 1` from a client — then `bt` and walk the frames you just read about.



# Chapter 2 — The Parser

SQL text → tokens → parse tree → LEX. The first transformation of a query's life.  
Source: `sql/sql_lex.cc`, `sql/sql_yacc.yy`, `sql/parse_tree_nodes.h/.cc`, `sql/sql_cmd.h`

## 2.1 Three representations, two steps

MySQL's front end turns a query string into three successive data structures:

```
"SELECT a FROM t WHERE b>1"
  | lexer: Lex_input_stream + MySQLlex()      (sql/sql_lex.cc:1310)
  ▼
tokens (SELECT_SYM, IDENT, FROM, IDENT, WHERE, IDENT, '>', NUM)
  | Bison grammar: sql/sql_yacc.yy (~18,500 lines)
  ▼
Parse Tree: PT_select_stmt → PT_query_expression → PT_query_specification → ...
  | contextualize(): PT_select_stmt::make_cmd() (sql/parse_tree_nodes.cc:707)
  ▼
LEX + Query_expression / Query_block + Item trees      ← what the rest of the server uses
  └─ plus an Sql_cmd object (e.g. Sql_cmd_select) that will drive prepare/execute
```

## 2.2 The lexer

`Lex_input_stream` (`sql/sql_lex.h:3227`) is a hand-written scanner: a cursor over the query buffer (`yyGet/yyPeek/yyUnget` primitives), a state machine in `lex_one_token()` (`sql/sql_lex.cc:1412`), keyword lookup, and two notable extras:

- **Digest computation on the fly** (`add_digest_token`, `sql/sql_lex.cc:386`): as tokens stream by, a normalized statement digest is built — this is where `performance_schema.events_statements_summary_by_digest` gets its query fingerprints. The parser and the observability system share one pass.
- **Token lookahead** for grammar warts: `MySQLlex()` (`sql_lex.cc:1310`) buffers one token to disambiguate cases like `WITH ROLLUP` (`:1383-1407`) — a reminder that SQL was not designed for LALR(1) parsers.

## 2.3 The grammar and the parse tree

`sql/sql_yacc.yy` is one of the largest Bison grammars in production use. Its actions do **no semantic work** — each rule just allocates a `PT_*` node on the parse arena (`NEW_PTN`). The `SELECT` chain, top-down:

```

select_stmt (:9933)          → PT_select_stmt
└─ query_expression (:10029) → PT_query_expression    (+ WITH clause = CTEs)
   └─ query_expression_body (:10045) UNION/EXCEPT/INTERSECT nesting
      └─ query_primary (:10085)
         └─ query_specification (:10106) → PT_query_specification
            SELECT options, item list, FROM, WHERE, GROUP BY, HAVING, windows

```

**Why the intermediate tree?** Older MySQL built `LEX` structures directly in grammar actions, which made the grammar unmaintainable (actions ran in grammar-reduction order, not semantic order) and blocked reuse. 8.0's two-phase design — *build a dumb tree, then walk it* — is the textbook approach, arrived at 20 years late and still being migrated command by command (look at how many `SQLCOM_*` cases in `mysql_execute_command` still don't go through `Sql_cmd`).

The walk is called **contextualization**: `PT_select_stmt::make_cmd()` (`sql/parse_tree_nodes.cc:707`) creates a `Parse_context` and calls `contextualize()` recursively. Each node populates the semantic structures — e.g. `PT_query_specification::contextualize()` (`:1170`) fills the current `Query_block`'s select list, table list, WHERE and HAVING. The output is an `Sql_cmd_select` stored in `lex->m_sql_cmd`, whose `prepare()/execute()` (Chapter 3) drive everything else.

## 2.4 The output structures: LEX, Query\_block, Item

Three families you must know to read any `sql/` code:

- **LEX** (`sql/sql_lex.h:3753`) — the per-statement descriptor: `sql_command` (the `SQLCOM_*` verb), the tree of query blocks, the parameter list, flags. One per statement, hung off `thd->lex`.
- **Query\_expression / Query\_block** (`sql/sql_lex.h:623, :1160`) — the query's shape. A *block* is one SELECT (fields, table list, where/having, group/order); an *expression* is one or more blocks combined by UNION/EXCEPT/INTERSECT plus ORDER/LIMIT. Subqueries nest: an inner `Query_expression` hangs under an `Item_subselect` in the outer block. Every later phase — resolver, optimizer, executor — is organized around walking this tree.
- **Item** (`sql/item.h:853`) — every expression node: `Item_field` (a column ref), `Item_func_*` (functions/operators), `Item_cond_and/or`, literals, `Item_param` (a `?` placeholder), `Item_subselect`. Items are created "unresolved" by the parser; the key virtual `fix_fields()` (`item.h:1180`) — run in the next chapter — binds names to real columns and derives types. An item can *replace itself* through the `Item**` reference parameter, which is how many rewrites are implemented.

All of it is allocated on the statement `MEM_ROOT` — no destructors, freed wholesale when the statement ends (or kept, for prepared statements — Chapter 3).

## 2.5 What to remember

1. Parsing is two phases: a Bison grammar building a dumb `PT_*` tree, then `contextualize()` producing the semantic `LEX/Query_block/Item` structures and an `Sql_cmd`. The split exists because the one-phase design collapsed under its own weight.
2. `Query_block` (one SELECT) and `Query_expression` (blocks + set operations) are the backbone data structure of the entire SQL layer — learn their fields once, use them in every chapter after.
3. `Item` trees are the universal expression currency, unresolved until `fix_fields()`.
4. Statement digests (your `performance_schema` fingerprints) are computed inside the lexer.

**Try it:** `EXPLAIN FORMAT=JSON` names query blocks (`query_block`, `nested_loop...`) exactly as they exist in this tree; and `SELECT STATEMENT_DIGEST_TEXT('SELECT a FROM t WHERE b>1');` shows the lexer's normalization directly.

---

**Previous:** [Chapter 1 — Connections & Dispatch](#) · **Next:** [Chapter 3 — Resolution & Prepare](#)

# Chapter 3 — Resolution & Prepare

Binding names to tables and columns, opening tables, and the permanent query transformations — the phase between parsing and optimization. Source: `sql/sql_resolver.cc`, `sql/sql_base.cc`, `sql/table.h`, `sql/sql_prepare.cc`

## 3.1 Where we are

The parser produced an `Sql_cmd` (Chapter 2). For DML, `Sql_cmd_dml::prepare()` (`sql/sql_select.cc:495`) now runs — once per statement, or once *ever* for a prepared statement:

```
Sql_cmd_dml::prepare
├ open_tables_for_query()      (sql/sql_base.cc:6829)  tables + MDL
└ prepare_inner → Query_block::prepare() (sql/sql_resolver.cc:180)
    name resolution + permanent transformations
```

## 3.2 Opening tables: shares, instances, references

Three layers of "table" (all `sql/table.h`) — confuse them and no server code makes sense:

| struct                                                   | one per                                                                     | contents                                                                   |
|----------------------------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <code>TABLE_SHARE</code><br>( <code>table.h:692</code> ) | table <i>definition</i> , process-wide, cached & refcounted                 | columns, keys, version — loaded from the data dictionary (Ch. 10)          |
| <code>TABLE</code> ( <code>table.h:1431</code> )         | <i>open instance</i> , per (session × table use)                            | the <code>handler*</code> file (Ch. 6), record buffers, read/write bitmaps |
| <code>Table_ref</code><br>( <code>table.h:2872</code> )  | <i>mention in the query</i> (FROM entry, formerly <code>TABLE_LIST</code> ) | name, alias, links into the query block, <code>mdl_request</code>          |

`open_tables_for_query()` walks the statement's `Table_ref` list: acquire the **metadata lock** (`Table_ref::mdl_request`, `table.h:3949` — DML takes `MDL_SHARED_READ/WRITE`; the MDL system is Chapter 7), find/load the `TABLE_SHARE`, get a `TABLE` from the per-session table cache, and point `Table_ref::table` at it. Views and derived tables expand here too.

## 3.3 `Query_block::prepare()` : name resolution

`sql/sql_resolver.cc:180` runs a fixed sequence per query block (recurring into subqueries):

1. `setup_tables()` (`:1175`) — assign table numbers/maps, build the leaf-table list.

2. `resolve_placeholder_tables()` (:1300) — resolve derived tables and views; **merge** mergeable ones into the parent block (`merge_derived`, :3473) so `SELECT * FROM (SELECT ...)` can often optimize as a plain join.
3. `setup_wild()` (:1629) — expand `*` into concrete `Item_fields`.
4. `setup_fields()` (`sql/sql_base.cc:9063`) — run `fix_fields()` on the select list: every `Item_field` finds its `Field` in an opened `TABLE`; types, nullability, collations are derived bottom-up. Ambiguous or unknown columns error here.
5. `setup_conds()` (:1685) — same for join conditions and `WHERE`.
6. `setup_group()` / `HAVING` / `setup_order()` / `resolve_limits` — with SQL's peculiar scoping (`HAVING` sees aliases, `ORDER BY` sees both...) encoded in `Name_resolution_context`.

### 3.4 Permanent transformations

Interleaved with resolution, the server performs rewrites that are **structural and irreversible** — done once, valid for every future execution (which is precisely why they live here and not in the optimizer):

- **Outer-join simplification** (`simplify_joins`, :1935): `LEFT JOIN` becomes inner join when the `WHERE` clause rejects `NULL`-complemented rows — the classic `... LEFT JOIN t2 ... WHERE t2.x = 5` case.
- **Subquery → semijoin** (`resolve_subquery` :1384, `flatten_subqueries` :3822, `convert_subquery_to_semijoin` :3009): `WHERE id IN (SELECT ...)` is pulled up into the outer block as a *semijoin nest*, letting the join optimizer choose among execution strategies (Chapter 4) instead of executing the subquery per row.
- **Scalar subquery → derived table** (`transform_scalar_subqueries_to_join_with_derived`, :7681) and `IN→EXISTS` injection for subqueries that can't be flattened.
- **Redundant clause removal** in subqueries (`remove_redundant_subquery_clauses`, :4151) — an `ORDER BY` inside an `IN`-subquery means nothing, so it's deleted.

After `prepare()`, the `Query_block` is fully bound and normalized. `Sql_cmd_dml::execute()` (`sql/sql_select.cc:663`) can now run it — going next to the optimizer.

### 3.5 Prepared statements: why this phase is separable

The prepare/execute split is not just an API nicety — it is *this* boundary:

- `Prepared_statement` (`sql/sql_prepare.h:346`) parses + resolves **once**, keeping the `LEX` and `Item` trees alive in their own `MEM_ROOT` (`m_arena`); ? placeholders are `Item_params` collected in `lex->param_list`.
- `execute_loop()` (`sql/sql_prepare.cc:2993`) binds parameter values and reruns optimization + execution against the saved trees.
- The catch: cached resolution can go **stale** (table altered, index dropped). Every open validates the `TABLE_SHARE` version; a mismatch raises `ER_NEED_REPREPARE`, caught by the `Reprepare_observer`, and `Prepared_statement::reprepare()` (:3216) transparently re-parses, re-

resolves, swaps in the new tree, and retries — invisible to the client.

This is also why the permanent-vs-per-execution split matters: transformations in this chapter survive across executions; everything the optimizer does (next chapter) is redone per execution.

## 3.6 What to remember

1. Three "table" structs: `TABLE_SHARE` (cached definition), `TABLE` (open instance with the engine handler), `Table_ref` (query mention holding the MDL request).
2. `fix_fields()` is where names become columns and types are derived — the moment SQL text semantics is finally pinned down.
3. Big rewrites (outer-join reduction, subquery flattening/semijoins, derived-table merging) happen here, permanently — the optimizer never sees the original shape.
4. Prepared statements = "freeze the output of this chapter, redo everything after it", with automatic re-prepare on metadata changes.

**Try it:** `EXPLAIN a WHERE x IN (SELECT ...)` query — the plan shows the semijoin (`FirstMatch/MaterializeLookup...`), proof the subquery ceased to exist in this phase.

---

**Previous:** [Chapter 2 — The Parser](#) · **Next:** [Chapter 4 — The Optimizer](#)

# Chapter 4 — The Query Optimizer

---

Cost-based planning: statistics, range analysis, join ordering, and the AccessPath tree that the executor will run. Source: `sql/sql_optimizer.cc`, `sql/sql_planner.cc`, `sql/range_optimizer/`, `sql/opt_costmodel.h`, `sql/join_optimizer/`

## 4.1 The pipeline: `JOIN::optimize()`

Per query block, `JOIN::optimize()` (`sql/sql_optimizer.cc:337`) runs a fixed pipeline:

optimize\_cond (:473)  
constant folding, equality  
propagation  
→ may prove 'Impossible  
WHERE'



prune\_table\_partitions  
(:518)



optimize\_aggregated\_query

(:533)

MIN/MAX/COUNT via index

peek — answer  
without executing



make\_join\_plan (:5350)

constant tables, range

analysis,

JOIN ORDER SEARCH

(4.3)



plan refinement:  
attach conditions per table  
(:797),  
avoid filesort via index  
(:1031),  
join buffering, tmp-table  
decisions



create\_access\_paths  
(:1053)  
→ AccessPath tree (4.4)

Two early-exit tricks worth knowing: `optimize_cond` can prove the WHERE false and skip everything (Impossible WHERE); `optimize_aggregated_query` can answer `SELECT MAX(k) FROM t` from one index dive with no execution at all.

## 4.2 Costs and statistics

Cost = abstract units where reading one disk block  $\approx 1.0$  (`opt_costconstants.cc`):

`ROW_EVALUATE_COST` = 0.1, `KEY_COMPARE_COST` = 0.05, `MEMORY_BLOCK_READ_COST` = 0.25,

`IO_BLOCK_READ_COST` = 1.0 — all overridable via the `mysql.server_cost` / `mysql.engine_cost` tables (a rarely-used but real tuning surface). `Cost_estimate` (`sql/handler.h:3799`) tracks io/cpu components separately.

The numbers being multiplied come from the storage engine, through three channels:

1. `handler::info()` (`sql/handler.h:5721`) — table row count (`stats.records`) and index cardinality (`rec_per_key`). For InnoDB these come from sampled index dives — *estimates*, *not counts*.
2. `handler::records_in_range()` (`:5681`) — "how many rows between key A and B?" answered by actual B+tree descents (index dives) at optimization time. Precise but costs I/O; `eq_range_index_dive_limit` caps how many ranges get dives before falling back to `rec_per_key`.
3. **Histograms** (`sql/histograms/`) — equi-height column distributions for predicates on non-indexed columns (`ANALYZE TABLE ... UPDATE HISTOGRAM`).

This is the layered answer to "where do optimizer estimates come from" — and the origin of most bad-plan war stories: stale dives, capped dives, or no histogram.

## 4.3 The two searches: range analysis and join ordering

**Range analysis** (`sql/range_optimizer/`, entry `test_quick_select`, `range_optimizer.cc:524`): the WHERE clause is compiled per index into a `SEL_TREE` of key-interval trees (`SEL_ARG` red-black trees — one of MySQL's oldest and hairiest data structures). Output: for each usable index, the set of ranges to scan and their estimated row count, possibly combined as index-merge / ROR union-intersection, or skip scans. Each candidate becomes a potential access method with a cost.

**Join ordering** (`sql/sql_planner.cc`) is the heart:

- `Optimize_table_order::choose_table_order()` (`:1959`) → `greedy_search()` (`:2337`) → `best_extension_by_limited_search()` (`:2736`): build the join prefix one table at a time; at each step, exhaustively evaluate extensions up to `optimizer_search_depth` tables deep, commit the best first table, repeat. Exhaustive within a window, greedy across — the classic answer to  $n!$  blowup (System R's DP, budgeted).
- For each candidate table position, `best_access_path()` (`:982`) picks the cheapest way to fetch its rows given the tables already in the prefix: `ref` lookup on an equality (from the `KEYUSE` array), a range scan from 4.2, a full index scan, or a table scan — costed with the prefix's row count as the multiplier.
- Semijoin strategies (from the flattened subqueries of Chapter 3) are costed alongside: `FirstMatch`, `LooseScan`, `DuplicateWeedout`, `Materialization` (`advance_sj_state`, `:4125`).

The output is `best_positions[]` — an ordered list of (table, access method, expected rows, cost).

**Left-deep plans only:** the classic optimizer never considers bushy joins.

## 4.4 The output: an AccessPath tree

Since 8.0.20+, the optimizer's product is an `AccessPath` tree (`sql/join_optimizer/access_path.h:193`) — a tagged union of plan nodes: `TABLE_SCAN`, `REF`, `EQ_REF`, `INDEX_RANGE_SCAN`, `NESTED_LOOP_JOIN`, `HASH_JOIN`, `FILTER`, `SORT`, `AGGREGATE`, `LIMIT_OFFSET`, `MATERIALIZATION`... each carrying cost and row estimates. `create_access_paths()` (`sql_optimizer.cc:1053`) translates the chosen `best_positions[]` into this tree. It is exactly what `EXPLAIN FORMAT=TREE` prints (`PrintQueryPlan`, `sql/join_optimizer/explain_access_path.cc:1727`), and what the executor consumes (Chapter 5).

### The future in the same tree: the hypergraph optimizer

`sql/join_optimizer/` also contains a complete second optimizer (`FindBestQueryPlan`, `join_optimizer.cc:6402`; entered from `JOIN::optimize` at `:638`): a textbook bottom-up **dynamic programming enumeration (DPhyp)** over a join hypergraph — bushy plans, principled cost model, interesting orders. In this 8.0 tree it's compile-gated (`WITH_HYPERGRAPH_OPTIMIZER`) and experimental; it became the default foundation in later MySQL versions. Having both in one codebase makes this tree a great place to compare 1990s-heuristic and modern-DP optimizer design side by side.

## 4.5 What to remember

1. Optimization = fixed pipeline: normalize conditions → prune → range-analyze each table → greedy join-order search with per-position `best_access_path` → `AccessPath` tree.
2. Estimates flow from the engine (`info`, `records_in_range` dives, histograms) into an abstract cost model with configurable constants. Most plan mysteries are estimate mysteries.
3. `optimizer_search_depth`, `optimizer_prune_level`, `eq_range_index_dive_limit`, `optimizer_switch` flags — the knobs map 1:1 to functions in this chapter.
4. The `AccessPath` tree is the contract between optimizer and executor — and `EXPLAIN FORMAT=TREE` is its honest serialization.

**Try it:** `SET optimizer_trace="enabled=on"; SELECT ...; SELECT * FROM information_schema.optimizer_trace\G` — the trace JSON mirrors this chapter's function names (`rows_estimation`, `considered_execution_plans`, `best_access_path`...).

---

**Previous:** [Chapter 3 — Resolution & Prepare](#) · **Next:** [Chapter 5 — The Iterator Executor](#)

# Chapter 5 — The Iterator Executor

How the plan actually runs: the volcano model, MySQL-style — plus filesort and internal temporary tables. Source: `sql/sql_union.cc`, `sql/iterators/`, `sql/join_optimizer/access_path.cc`, `sql/filesort.cc`, `sql/sql_tmp_table.cc`

## 5.1 The execution loop is ten lines

Since 8.0.20+ MySQL executes every query the same way

(`Query_expression::ExecuteIteratorQuery`, `sql/sql_union.cc:1682`):

```
m_root_iterator->Init();           // :1779
for (;;) {
    int error = m_root_iterator->Read(); // 0 = row, -1 = EOF, 1 = error
    if (error) break;
    query_result->send_data(thd, *fields); // row is in table->record[0] / Items
}
query_result->send_eof(thd);
```

Everything a query does — joins, aggregation, sorting, subqueries — is inside that `Read()` call, in a tree of **iterators**. This replaced the legacy pre-8.0.20 executor (a fixed nested-loop machine with special cases braided through it) with the textbook Volcano model: each operator pulls rows from its children.

## 5.2 The RowIterator contract and the zoo

`RowIterator` (`sql/iterators/row_iterator.h:82`) is a minimal interface: `Init()` (may be called again to rewind — that's how the inner side of a nested loop restarts), `Read()` (produce one row into the table record buffers), plus `SetNullRowFlag` (outer-join NULL rows) and `UnlockRow` (release a lock on a row that failed the WHERE — the MVCC/locking interplay from the InnoDB series surfacing here).

Iterators come in three families (`sql/iterators/`):

| family | examples                                                                                                                                                                                                                                                                                                   | role                                   |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| leaf   | <code>TableScanIterator</code> ( <code>basic_row_iterators.h:56</code> ), <code>IndexScanIterator</code> , <code>RefIterator</code> , <code>EQRefIterator</code> , <code>IndexRangeScanIterator</code> , <code>ConstIterator</code>                                                                        | fetch rows via the handler API (Ch. 6) |
| join   | <code>NestedLoopIterator</code> ( <code>composite_iterators.h:323</code> ), <code>HashJoinIterator</code> ( <code>hash_join_iterator.h:253</code> ), <code>BKAIterator</code>                                                                                                                              | combine two children                   |
| shape  | <code>FilterIterator</code> , <code>SortingIterator</code> , <code>AggregateIterator</code> , <code>LimitOffsetIterator</code> , <code>MaterializeIterator</code> ( <code>composite_iterators.cc:621</code> ), <code>StreamingIterator</code> , <code>WeedoutIterator</code> , <code>WindowIterator</code> | transform a stream                     |

`CreateIteratorFromAccessPath()` (`sql/join_optimizer/access_path.cc:378`) maps the optimizer's `AccessPath` tree 1:1 onto iterators — a giant switch, deliberately iterative rather than recursive to bound stack depth. `EXPLAIN FORMAT=TREE` output *is* this tree; `EXPLAIN ANALYZE` wraps every node in a `TimingIterator` and prints actual rows/timings.

A `SELECT ... JOIN ... WHERE ... ORDER BY` becomes, literally:

```

SortingIterator
└─ NestedLoopIterator
   └─ FilterIterator( t1.x > 10 )
      └─ TableScanIterator(t1)
         └─ RefIterator(t2, key=t1.id)      ← Init() re-run per t1 row

```

**Hash join** (8.0.18+) deserves a note: `HashJoinIterator` builds an in-memory hash table from one side (spilling to disk chunks when `join_buffer_size` is exceeded) and probes with the other — finally giving MySQL a join that doesn't need an index, and quietly replacing the old "block nested loop" buffering.

## 5.3 filesort: ORDER BY without an index

When no index provides the order, `SortingIterator` wraps **filesort** (`filesort(...)`, `sql/filesort.cc:367`) — a classic external sort:

1. Pull all rows from the child iterator, building fixed-format **sort keys** (`make_sortkey`) in a buffer of `sort_buffer_size`.
2. Buffer full → sort it, write a run (`Merge_chunk`) to a temp file; repeat.
3. Merge runs (`merge_many_buff` → `merge_buffers`, :1918) down to one sorted stream.

The payload question — what travels with the key — has two answers (`Addon_fields_status`, `sql/sort_param.h:49`): **addon fields** (pack the needed columns with the key; no second table visit) or **row IDs** (sort small entries, then re-fetch each row — required when rows are too wide or contain big BLOBs). That choice is the difference between one and two passes over the data, and you can observe it in `optimizer_trace`'s `filesort_summary`.

## 5.4 Internal temporary tables

`MaterializeIterator`, `TempTableAggregateIterator`, window buffering, UNION/CTE materialization — all need scratch tables (`create_tmp_table`, `sql/sql_tmp_table.cc:886`). The engine ladder (`setup_tmp_table_handler`, `:2102`):

```
in-memory: TempTable engine (storage/temptable/, default; supports VARCHAR/BLOB)
           - or legacy MEMORY engine (internal_tmp_mem_storage_engine)
           | exceeds temptable_max_ram / row constraints (use_tmp_disk_storage_engine :2044)
           ▼
on-disk: InnoDB temp tablespace (ibtmp1)
```

"Using temporary; Using filesort" in EXPLAIN — the two most famous words in MySQL performance work — refer precisely to sections 5.3 and 5.4.

## 5.5 What to remember

1. One execution model: `AccessPath` → iterator tree → `Init()/Read()` pull loop. Rows materialize in `table->record[0]` buffers; `Query_result` streams them to the client as produced (Ch. 1).
2. The iterator names in `EXPLAIN FORMAT=TREE` are literal C++ classes in `sql/iterators/` — the plan you read is the code that runs.
3. filesort = external merge sort; `addon-fields` vs `row-id` decides whether sorted rows must be re-fetched. Hash join covers index-less equijoins with disk spill.
4. Temp tables cascade `TempTable` → InnoDB-on-disk; most "why is this query slow" answers live in 5.3/5.4.

**Try it:** `EXPLAIN ANALYZE SELECT ... ORDER BY ...` — match every line to an iterator class, then find its `Read()` in `sql/iterators/` and read it. Most are under a page.

---

**Previous:** [Chapter 4 — The Optimizer](#) · **Next:** [Chapter 6 — The Handler API](#)

# Chapter 6 — The Handler API: Where SQL Meets the Storage Engine

The pluggable-engine interface — and the exact point where this series hands off to the InnoDB deep-dive. Source: `sql/handler.h`, `sql/handler.cc`, `storage/innobase/handler/ha_innodb.cc`

## 6.1 Two objects define an engine

MySQL's storage-engine pluggability rests on one pair (`sql/handler.h`):

- `handlerton` (:2663) — "handler singleton": one per engine, a struct of function pointers for everything *not* tied to one table: `commit`, `rollback`, `prepare` (the 2PC surface — Chapter 7), `create` (the handler factory), DDL hooks, and the data-dictionary bridge (`ddse_dict_init` — InnoDB serving as dictionary engine, Chapter 10).
- `class handler` (:4506) — one per *open table per session*: a cursor-style abstract class the executor drives.

The core `handler` virtuals form a tiny, 1990s-flavored ISAM interface:

```
lifecycle  open() close() external_lock() start_stmt()
full scan  rnd_init() rnd_next(buf) rnd_pos(buf, rowid)
index scan index_init(idx) index_read_map(key) index_next() index_next_same()
writes     write_row(buf) update_row(old,new) delete_row(buf)
metadata   info(flags) records_in_range(idx, min, max) position(record)
```

Every iterator from Chapter 5 bottoms out here: `TableScanIterator::Read()` calls `ha_rnd_next`; `RefIterator` calls `ha_index_read_map`; the row-apply path of replication (Ch. 9) calls `ha_write_row`. The public `ha_*` wrappers (e.g. `handler::ha_write_row`, `sql/handler.cc:8353`) add PSI instrumentation, then invoke the engine's virtual — and on success call `binlog_log_row()`: **row-based binlogging is a side effect of the handler API**, which is why it works identically for every engine.

Rows cross the boundary in **MySQL row format**: `TABLE::record[0]` (current row) and `record[1]` (old row for updates) — `table.h:1486`. The engine converts to its own format internally.

An engine registers by declaring a plugin (`mysql_declare_plugin(innobase)`, `ha_innodb.cc:24586`) whose init function fills in the `handlerton` (`innodb_init`, :5526: `->commit = innobase_commit`, `->create = innobase_create_handler`, ...). "Storage engine" and "plugin" are the same mechanism.

## 6.2 Crossing the bridge into InnoDB

`ha_innobase` (`storage/innobase/handler/ha_innodb.cc`) implements the interface, and if you did the InnoDB series, everything behind it is familiar — the same machinery, 20 years after the embedded version:

| handler call                                                             | InnoDB internal                                                                                      | InnoDB series                                |
|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------|
| <code>open()</code> (:7716)                                              | <code>row_create_prebuilt()</code> →<br><code>row_prebuilt_t</code> ( <code>row0mysql.h:578</code> ) | the <code>ib_csr_t</code> prebuilt (Ch. 11)  |
| <code>write_row()</code> (:9587)                                         | <code>row_insert_for_mysql()</code> → insert graph →<br><code>row_ins</code>                         | <code>ib_cursor_insert_row</code> (Ch. 9.2)  |
| <code>index_read()</code> (:10802)                                       | <code>row_search_mvcc()</code><br>( <code>row/row0sel.cc:4297</code> )                               | <code>row_search_for_client</code> (Ch. 9.3) |
| <code>rnd_next()</code> (:11473)                                         | <code>general_fetch</code> → <code>row_search_mvcc</code>                                            | same                                         |
| <code>update_row()</code> / <code>delete_row()</code><br>(:10348/:10514) | <code>row_update_for_mysql()</code>                                                                  | <code>row_upd</code> , delete-mark (Ch. 9.4) |
| <code>records_in_range()</code> (:17403)                                 | <code>btr_estimate_n_rows_in_range</code> — B+tree<br>dives                                          | the index dives feeding Ch. 4's optimizer    |
| <code>info()</code> (:18232)                                             | sampled statistics → <code>stats.records</code> ,<br><code>rec_per_key</code>                        | <code>dict_index_t stats</code> (Ch. 10)     |

The **row template** mechanism (`build_template()`, `ha_innodb.cc:8957`) precompiles the column mapping between InnoDB records and `record[0]` into `mysql_row_tmpl_t[]`, so `row_sel_store_mysql_rec()` (`row/row0sel.cc:2678`) can convert each fetched row with memcpy-level work rather than re-interpreting metadata per row. The `row_prebuilt_t` caching this — along with the persistent cursor it embeds — is the *same struct by the same name* as in Embedded InnoDB: the handler API is, in a real sense, the embedded API with MySQL as the only client.

## 6.3 The impedance mismatches

The interface's simplicity has costs, and much of MySQL's evolution is visible as patches around them:

- **Row-at-a-time:** one virtual call (and engine latch dance) per row. Mitigations: MRR (`multi_range_read_*` — batch ranges, sort rowids), batched key access, and the handler's ICP hook (`index_read_pushed`) that pushes WHERE fragments *into* the engine's index scan, filtering before row materialization.
- **Opaque statistics:** the optimizer sees only `info()/records_in_range()` — estimate quality is engine-private (InnoDB's sampled dives), invisible to the planner.
- **Transaction coordination:** each engine commits independently, so multi-engine (or engine + binlog) atomicity needs the server-level 2PC — the next chapter.

Compare MyRocks ( `storage/rocksdb/ha_rocksdb.cc` — Chapter 11): a completely different storage architecture (LSM tree) behind the identical interface. That's the payoff of the abstraction, mismatches and all.

## 6.4 What to remember

1. `handlerton` = engine-level ops (commit/prepare/create); `handler` = per-open-table cursor with an ISAM-shaped API. Iterators above, pages and B+trees below.
2. Rows cross in MySQL record format; InnoDB's prebuilt + row template make the conversion cheap. `ha_*` wrappers add instrumentation and row-based binlogging.
3. `ha_innabase` maps almost 1:1 onto the row operations of the InnoDB deep-dive — `row_search_mvcc` is `row_search_for_client` grown up.
4. The API's row-at-a-time, stats-opaque design explains ICP, MRR, BKA, and the whole "pushdown" genre of MySQL features.

**Try it:** in gdb, `break ha_innabase::index_read` and run a point SELECT; walk the stack up (RefIterator → executor loop) and down ( `row_search_mvcc` ) — the two halves of both series meet in that one backtrace.

---

**Previous:** [Chapter 5 — The Iterator Executor](#) · **Next:** [Chapter 7 — MDL & Transaction Coordination](#)

# Chapter 7 — Metadata Locks & Transaction Coordination

The two server-level concurrency systems: MDL, which serializes DDL against DML, and the two-phase commit that keeps engines and binlog atomically in sync. Source: `sql/mdl.h/.cc`, `sql/handler.cc`, `sql/transaction_info.h`, `sql/tc_log.h`

## 7.1 Why the server needs its own locks

InnoDB locks *rows* (InnoDB series, Ch. 8). But nothing at the engine level stops `DROP TABLE t` from destroying a table mid-`SELECT`. The server needs locks on *names and definitions* — **metadata locks (MDL)**, held for the whole transaction (that duration is the fix for the historical bug where a table could change between statements of one transaction).

MDL\_key namespaces (`sql/mdl.h:401`) cover more than tables: `GLOBAL`, `SCHEMA`, `TABLE`, `COMMIT`, `TABLESPACE`, `FUNCTION`... (plus Percona's `BACKUP_TABLES` for `LOCK TABLES FOR BACKUP`). Lock types run from `MDL_INTENTION_EXCLUSIVE` through the shared family to `MDL_EXCLUSIVE (:197)`:

```
DML:  SELECT → SHARED_READ (SR) on the table    INSERT/UPDATE/DELETE → SHARED_WRITE (SW)
      + INTENTION_EXCLUSIVE on GLOBAL, SCHEMA, COMMIT scopes
DDL:  SHARED_UPGRADABLE (SU) while preparing → upgrade to EXCLUSIVE (X) to swap the definition
FTWRL (backup): GLOBAL S – blocks all IX holders = all writes
```

SR and SW are compatible with each other (DML never blocks DML at this level) but conflict with X — so a `DROP/ALTER` waits for open transactions, and everyone behind it waits on the X request. That queue is the anatomy of the infamous **"Waiting for table metadata lock"** pile-up: one long transaction + one `ALTER` = a frozen table.

Acquisition (`MDL_context::acquire_lock`, `sql/mdl.cc:3378`) waits up to `lock_wait_timeout`, and — because transactions can wait on each other through MDL alone — runs a full **deadlock detector**: a wait-for-graph visitor (`Deadlock_detection_visitor`, `mdl.cc:301`; `find_deadlock`, `:4096`) that spans MDL *and* table locks, choosing the victim by lightest weight. This is a second, independent deadlock detector above InnoDB's (Ch. 8 there) — a deadlock can be caught by either, depending on where the cycle forms.

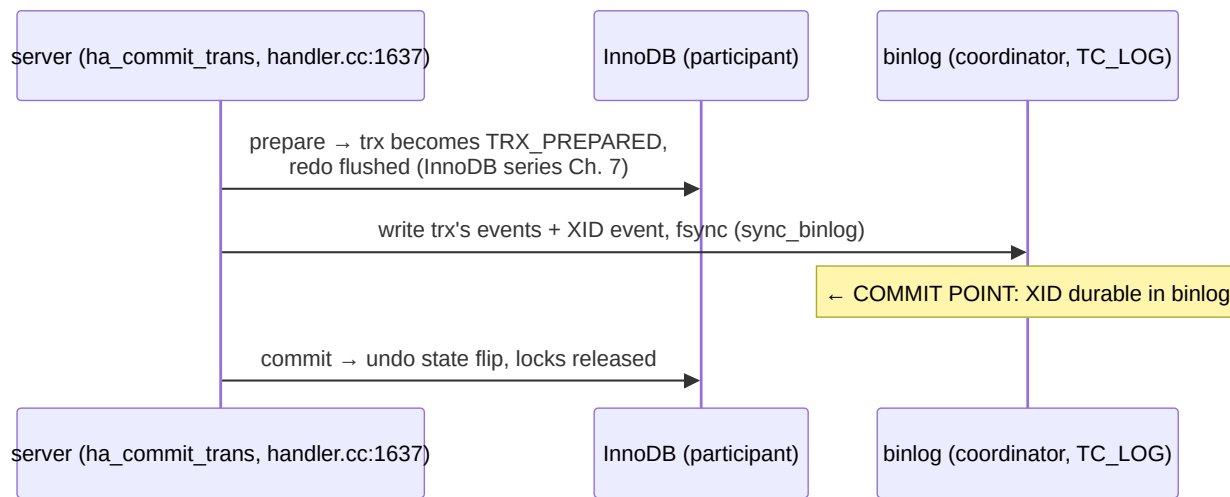
## 7.2 The intention lock on COMMIT

A detail that shows the design's elegance: every commit takes `MDL_INTENTION_EXCLUSIVE` on the **COMMIT namespace** (`ha_commit_trans`, `sql/handler.cc:1774`). `FLUSH TABLES WITH READ LOCK` takes the conflicting global lock — so a backup can block *commits* (not just statements) and get a true

point-in-time cut, without any special-case code in the commit path.

## 7.3 The distributed transaction inside every MySQL server

Here is the deepest fact of this chapter: with the binlog enabled, **every transaction is a distributed transaction** between two persistence systems — the storage engine (InnoDB's redo/undo) and the binary log (replication's source of truth, Ch. 8). If they disagree after a crash, replicas diverge from the source. MySQL solves it with classic two-phase commit, with the **binlog acting as the coordinator's log**:



Crash recovery (`binlog::Binlog_recovery::recover`, `sql/binlog/recovery.cc:59`) makes the rule concrete: scan the last binlog for XID events, then `ha_recover()` — every engine lists its prepared transactions; **in the binlog → commit it; not in the binlog → roll it back** (`recovery.cc:111`). The InnoDB `TRX_PREPARED` state you met in the embedded engine exists *for this handshake*. The binlog is also truncated back to the last complete transaction (`m_valid_pos`), handling its own torn tail.

The plumbing (`sql/handler.cc:1637`, `ha_commit_trans`):

- `Transaction_ctx` (`sql/transaction_info.h:53`) tracks which engines joined the transaction (`Ha_trx_info` list, `rw_ha_count`), per scope — statement vs session (that duality is how autocommit and multi-statement transactions share one code path).
- The prepare phase is skipped when there's only one participant (`handler.cc:1795`: read-only, single engine + no binlog) — then it's a plain 1PC commit.
- `tc_log` is the coordinator abstraction (`sql/tc_log.h`): `MYSQL_BIN_LOG` when binlogging; `TC_LOG_MMAP` (an mmap'd XID file) for binlog-less multi-engine XA; `TC_LOG_DUMMY` otherwise (`mysqld.cc:7281-7283`).

User-level `XA PREPARE/COMMIT` is the same machinery with the client as coordinator, and GTIDs piggyback on the commit path too (Ch. 8).

## 7.4 What to remember

1. MDL locks *definitions*, for transaction duration, with its own wait-for-graph deadlock detector; "Waiting for table metadata lock" = a DDL X request queued behind an open transaction.
2. The COMMIT-namespaces intention lock is how FTWRL/backups quiesce commits cleanly.
3. Binlog + engine = 2PC with the binlog as coordinator log; recovery commits prepared engine transactions iff their XID reached the binlog. InnoDB's `TRX_PREPARED` exists for exactly this.
4. `Transaction_ctx` / `Ha_trx_info` track per-engine participation; 2PC is skipped when only one durable participant exists.

**Try it:** reproduce the classic pile-up — session 1: `BEGIN; SELECT * FROM t;` session 2: `ALTER TABLE t ADD COLUMN x INT;` session 3: `SELECT * FROM t;` — then examine `performance_schema.metadata_locks` to see SR granted, X pending, SR waiting.

---

**Previous:** [Chapter 6 — The Handler API](#) · **Next:** [Chapter 8 — The Binary Log](#)

# Chapter 8 — The Binary Log

MySQL's second write-ahead log: the replication stream, its group commit, and GTIDs.  
Source: `sql/binlog.cc`, `libbinlogevents/`, `sql/rpl_gtid*.h`, `sql/rpl_commit_stage_manager.h`

## 8.1 Why a second log?

InnoDB already has a redo log (InnoDB series, Ch. 5) — but it's physical (page-level), engine-private, and circular. Replication needs the opposite: a **logical, engine-neutral, append-only** record of committed transactions that other servers can replay. That's the binlog. Every write is thus logged twice (redo + binlog) — the price of the pluggable-engine architecture, kept consistent by the 2PC of Chapter 7.

## 8.2 Events on the wire

A binlog file is a sequence of typed **events** (`libbinlogevents/`), each with a 19-byte header (`LOG_EVENT_HEADER_LEN`, `binlog_event.h:433`): timestamp, type, server\_id (the loop breaker for circular replication), length, next-position, flags — plus an optional CRC32.

One row-format transaction on disk:

|                       |                                                                  |
|-----------------------|------------------------------------------------------------------|
| GTID_LOG_EVENT        | "this transaction is UUID:42" (+ last_committed/sequence_number  |
| TABLE_MAP_EVENT       | table id → schema/table + column types for parallel apply, Ch.9) |
| WRITE_ROWS_EVENT      | before/after row images (binary)                                 |
| UPDATE_ROWS_EVENT ... |                                                                  |
| XID_EVENT             | commit marker = InnoDB XID ← the 2PC commit point of Ch. 7       |

`binlog_format` (ROW default, STATEMENT, MIXED): row events (`Rows_log_event`, generated at `ha_write_row` time via `THD::binlog_write_row`, `sql/binlog.cc:11531`) replay deterministically; statement events are compact but non-deterministic statements (UUID(), LIMIT without ORDER BY...) made STATEMENT mode a long tale of replication drift — the reason ROW won.

During execution, events accumulate in a per-THD **binlog cache** (`binlog_cache_data`, `sql/binlog.cc:718` — memory up to `binlog_cache_size`, then a temp file). Nothing touches the real binlog until commit, so transactions appear in the binlog **atomically and in commit order**.

## 8.3 Group commit: the three-stage pipeline

At commit, the transaction's cache must be flushed to the binlog file and fsynced — naively, one fsync per commit, on top of InnoDB's. MySQL 5.6+ solved this with a staged pipeline (`MYSQL_BIN_LOG::ordered_commit`, `sql/binlog.cc:9203`; stages in `Commit_stage_manager`,

```
rpl_commit_stage_manager.h:166):
```

```
committing sessions queue up; the FIRST in each stage's queue becomes the LEADER,  
processes EVERYONE queued, while the others sleep:
```

```
FLUSH stage    leader writes all queued caches to the binlog file  
                (+ assigns GTIDs to the whole group, binlog.cc:8813)  
SYNC stage     ONE fsync covers the whole group (sync_binlog=1)  
                [optional wait: binlog_group_commit_sync_delay to grow the group]  
COMMIT stage    engine commits run in queue order (binlog_order_commits)
```

This is the same group-commit idea the embedded InnoDB had for its redo log (InnoDB series Ch. 5) — reinvented at the server layer, with an explicit leader/follower queue instead of "piggyback on the running write". One fsync amortized over N transactions; `sync_binlog=1` + `innodb_flush_log_at_trx_commit=1` ("dual-1") became affordable because of this pipeline. Commit order is preserved end-to-end, which Chapter 9's parallel replication depends on.

## 8.4 GTIDs: names for transactions

A file:offset position is server-local and fragile. **GTIDs** name every transaction globally:

`server_uuid:sequence` (Gtid, `sql/rpl_gtid.h:1066`), assigned to the whole group during the flush stage (`Gtid_state::generate_automatic_gtid`, `sql/rpl_gtid_state.cc:488`). The server tracks sets of them (`Gtid_set` — interval lists, so `uuid:1-4000000` is two words): `gtid_executed`, `gtid_purged`, persisted in the `mysql.gtid_executed` table (`rpl_gtid_persist.h`).

The payoff is in Chapter 9: a replica says "I have executed *this set*" and the source streams everything else — no positions, safe failover, idempotent reapply. Auto-positioning replication, group replication, and every modern MySQL HA tool stand on this.

## 8.5 What to remember

1. Redo log = physical, engine, crash recovery; binlog = logical, server, replication & point-in-time recovery. Two logs, married by the XID 2PC.
2. A transaction = GTID + TABLE\_MAP + row events + XID, buffered per-session and appended atomically in commit order.
3. Group commit = leader/follower FLUSH→SYNC→COMMIT pipeline; one fsync per *group* makes full durability practical.
4. GTID sets turn replication state from "file+offset" into set algebra.

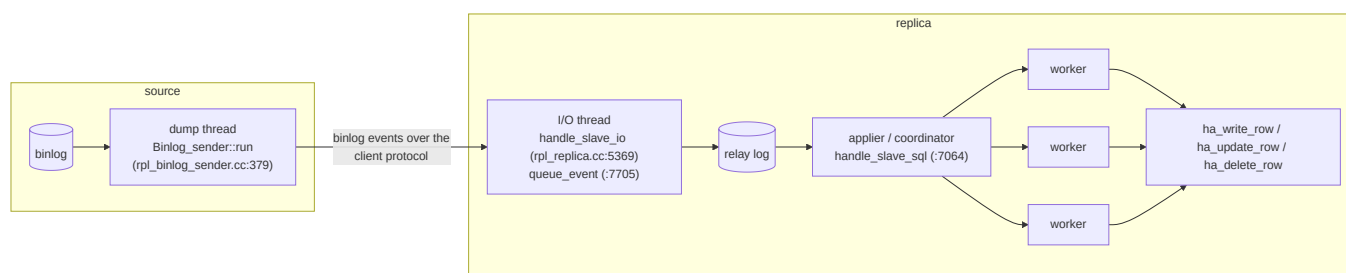
**Try it:** `SHOW BINLOG EVENTS`, then `mysqlbinlog --hexdump --verbose <file>` — find the 19-byte header fields and the row images; `SELECT @@gtid_executed` to see the set notation.

# Chapter 9 — Replication

How replicas stay in sync: the dump thread, the I/O and applier threads, parallel apply, and crash-safe positioning. Source: `sql/rpl_binlog_sender.cc`, `sql/rpl_replica.cc`, `sql/rpl_rli_pdb.h`, `sql/log_event.cc`, `plugin/semisync/`

## 9.1 The pipeline: three threads and two logs

MySQL replication is asynchronous log shipping, built from pieces you already know:



- A replica is **just a client** (Ch. 1): its I/O thread connects and sends `COM_BINLOG_DUMP_GTID` with its `gtid_executed` set (`request_dump`, `rpl_replica.cc:4178`); the source's dump thread streams events it lacks, waiting on the binlog's update signal for new ones.
- The I/O thread only *stores* events into the **relay log** (same format as the binlog) — decoupling network from apply.
- The applier executes events through the very stack of Chapters 5-6: `Rows_log_event::do_apply_event` (`sql/log_event.cc:9820`) opens the table by id (from `TABLE_MAP`), then per row calls `ha_write_row / ha_update_row(record[1], record[0]) / ha_delete_row (:12465-12614)`. Row-based apply is literally handler calls replayed — no SQL, no parser, no optimizer.

## 9.2 Parallel apply (MTS)

A single applier thread cannot keep up with a source committing on many connections. With `replica_parallel_workers > 0`, the applier becomes a **coordinator + workers** (`Slave_worker`, `sql/rpl_rli_pdb.h:498`), and the question becomes: *which transactions may run in parallel?* Two policies (`Mts_submode`, `sql/rpl_mta_submode.h:47`):

- `DATABASE` — transactions touching different schemas are independent. Coarse.
- `LOGICAL_CLOCK` — the source annotates each GTID event with (`last_committed`, `sequence_number`): transactions that were in the **same binlog group commit** (Ch. 8) proved they didn't conflict — they held their locks simultaneously — so the replica may apply them concurrently (`Mts_submode_logical_clock`, `rpl_mta_submode.h:129`).

That is a beautiful architectural echo: **group commit, built to save fsyncs, doubles as a conflict oracle for parallel replication.** (Later MySQL sharpened it with writeset tracking, deriving independence from row hashes rather than commit timing.)

`replica_preserve_commit_order` keeps workers committing in source order so the replica never exposes a state the source never had.

## 9.3 Crash safety and semi-sync

Two refinements turned replication from fragile to dependable:

- **Crash-safe positions:** the replica's progress (`mysql.slave_relay_log_info`) is an InnoDB table updated *in the same transaction* as the applied changes — position and data commit atomically (the same trick as the `mysql.gtid_executed` table, and philosophically the same as InnoDB storing its dictionary in itself). A crashed replica resumes exactly where it stopped; with GTID auto-positioning it can even reconstruct from the data alone.
- **Semi-synchronous replication** (`plugin/semisync/`): pure async means an acknowledged commit can vanish if the source dies. With semi-sync, after the binlog fsync (the SYNC stage of Ch. 8) the committing session waits — `ReplSemiSyncMaster::commitTrx` (`semisync_source.cc:644`) — until at least one replica acknowledges receipt (AFTER\_SYNC: ack before the engine commit becomes visible, so no client ever sees a state that could be lost), or a timeout degrades to async. This is the primitive under most MySQL HA setups; its consensus-flavored big sibling is Group Replication.

## 9.4 What to remember

1. Replication = binlog shipping: dump thread → I/O thread → relay log → applier. Row events replay as raw handler calls — replication is a client of Chapter 6, not of SQL.
2. GTID auto-positioning replaced file+offset coordination with set difference.
3. Parallel apply trusts the group-commit clock (`last_committed/sequence_number`): commit-group membership = proven non-conflict.
4. Positions live in InnoDB tables updated transactionally with the data — crash-safety by construction; semi-sync closes the async durability gap at the cost of one network RTT.

**Try it:** `SHOW REPLICATION STATUS\G` maps 1:1 to this pipeline (I/O vs SQL thread state, `Retrieved_Gtid_Set` VS `Executed_Gtid_Set`); `performance_schema.replication_applier_status_by_worker` shows the workers and their `last_committed` scheduling.

---

**Previous:** [Chapter 8 — The Binary Log](#) · **Next:** [Chapter 10 — The Data Dictionary](#)

# Chapter 10 — The Data Dictionary & Atomic DDL

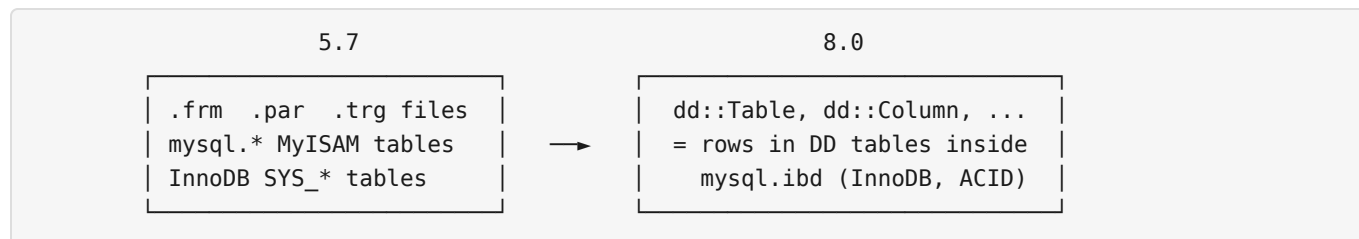
Where the schema lives since MySQL 8.0 — and how DDL finally became crash-safe.

Source: `sql/dd/`, `sql/sql_table.cc`, `storage/innobase/log/log0ddl.cc`,  
`storage/innobase/handler/handler0alter.cc`

## 10.1 The 8.0 revolution: no more .frm files

Before 8.0, MySQL's schema was scattered: `.frm` files for table definitions, `.par` for partitions, `.trg` for triggers, plus the engine's *own* dictionary (InnoDB's `SYS_TABLES` — the very tables you studied in the InnoDB series, Ch. 10). Two sources of truth, updated non-transactionally: a crash mid-DDL could leave an orphaned `.frm`, an InnoDB table with no `.frm`, or both halves disagreeing.

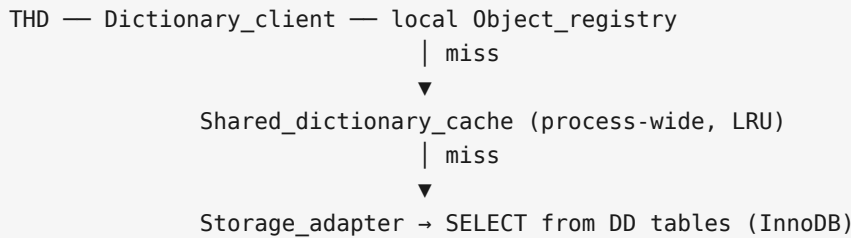
MySQL 8.0 deleted all of it. Now there is **one data dictionary, stored in InnoDB tables** (in the `mysql.ibd` tablespace), updated transactionally. The design InnoDB pioneered in 2000 — "the schema is just rows in my own B+trees" — was promoted to the whole server.



## 10.2 The `dd::` object model

The dictionary is exposed to server code as C++ objects (`sql/dd/types/`): `dd::Schema` → `dd::Table` (→ `dd::Column`, `dd::Index`, `dd::Foreign_key`, `dd::Partition`, `dd::Check_constraint`), plus `dd::Tablespace`, `dd::View`, `dd::Routine`, `dd::Trigger`, `dd::Event`... All descend from `dd::Entity_object` (id + name). Each type maps to a physical DD table declared in `sql/dd/impl/tables/` (`tables.cc`, `columns.cc`, `indexes.cc`, `schemata.cc`...). The dictionary's own version is a row in `dd_properties` (`DD_VERSION` = 80023, `sql/dd/dd_version.h`).

Access goes through a per-session `Dictionary_client` (`sql/dd/cache/dictionary_client.h:149`) backed by a three-tier cache:



The API is deliberately transactional-looking: `acquire()` (shared, read-only), `acquire_for_modification()` (returns a mutable clone), `store()/update()/drop()` (:1168-1224) — dictionary writes are just InnoDB row changes inside the DDL's transaction. An `RAII Auto_releaser` (:177) unpins acquired objects at scope exit.

Bootstrap solves the same chicken-and-egg the InnoDB series met at page 7: on first start, `dd::bootstrap::initialize()` (`sql/dd/impl/bootstrap/bootstrapper.cc:918`) creates the DD tablespace and tables from compiled-in definitions, then stores *the dictionary's own definition* into it (`populate_tables`, :595).

**INFORMATION\_SCHEMA fell out for free:** in 8.0 most `I_S` tables are just **SQL views over the DD tables** (`sql/dd/impl/system_views/` — one file per view). No more special-purpose C++ fill functions; `SELECT FROM information_schema.tables` is a plain query the optimizer can optimize. Percona adds two views of its own here (`compression_dictionary*.cc`).

## 10.3 Atomic DDL: the two-dictionary problem, solved properly

Even with one dictionary, DDL touches things a transaction cannot roll back: files created, B+trees built, tablespaces renamed. 8.0's answer has two coordinated halves:

1. **All metadata changes are one InnoDB transaction** — the DD rows (via `Dictionary_client`) and InnoDB's internal dictionary updates commit or roll back together.
2. **Physical file operations go through the InnoDB DDL log** — the `mysql.innodb_ddl_log` table (`storage/innobase/log/log0ddl.cc`; `DDL_Record` :78, `Log_DDL` manager :408). Before each irreversible physical step (create/free an index tree, delete a file, rename a tablespace), a compensating record is logged *in the same transactional world*:

|                                        |                                    |
|----------------------------------------|------------------------------------|
| CREATE TABLE t:                        | on crash:                          |
| ddl_log += "FREE index tree X"         | trx did not commit →               |
| build index tree X                     | replay ddl_log entries of the trx: |
| ddl_log += "DELETE file t.ibd"         | free tree X, delete t.ibd          |
| create t.ibd                           | → as if CREATE never happened      |
| insert DD rows (dd::Table, columns...) | trx committed →                    |
| COMMIT → post_ddl: purge the ddl_log   | nothing to do; ddl_log purged      |

So `DROP TABLE` on a crashing server can no longer strand half a table: recovery replays or discards the DDL log entries according to whether the owning transaction committed (`Log_DDL::replay_*`, `post_ddl`). This is the server-wide generalization of the undo-then-redo

layering from the InnoDB series (Ch. 5).

The SQL-layer flow for `CREATE TABLE` ties it together: `mysql_create_table` (`sql/sql_table.cc:10239`) → `create_table_impl` (`:8901`) → `rea_create_base_table` (`:1084`) → build the `dd::Table` object (`dd::create_dd_user_table`, `sql/dd/dd_table.h:88`) → `handler::create()` in the engine → one commit.

## 10.4 ALTER TABLE: COPY, INPLACE, INSTANT

Three algorithms, negotiated between server and engine

(`handler::check_if_supported_inplace_alter`, `storage/innobase/handler/handler0alter.cc`):

| algorithm      | what happens                                                                                                                                                                                                                                 | blocking       |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <b>COPY</b>    | server-level: create temp table, copy every row, swap ( <code>mysql_alter_table</code> , <code>sql/sql_table.cc</code> )                                                                                                                     | writes blocked |
| <b>INPLACE</b> | engine rebuilds/builds indexes itself, concurrent DML journaled and replayed ( <code>prepare/inplace/commit_inplace_alter_table</code> ; parallel builders in <code>storage/innobase/ddl/</code> )                                           | mostly online  |
| <b>INSTANT</b> | metadata-only — e.g. <code>ADD COLUMN</code> just records a new row version in the DD ( <code>Instant_Type</code> , <code>handler0alter.cc:913</code> ; version keys <code>DD_INSTANT_VERSION_ADDED/DROPPED</code> , <code>:515-581</code> ) | none           |

INSTANT is the endpoint of an old idea: since rows carry format metadata, new columns can exist only in the dictionary until a row is next rewritten — schema change in  $O(1)$ . (The row-versioning bookkeeping lives in the same DD private data that Ch. 2 of the InnoDB series would have called "the record format's evolution problem".)

## 10.5 What to remember

1. Since 8.0 the schema is **rows in InnoDB tables** (`mysql.ibd`), accessed through the `dd::` object model + a three-tier cache, versioned and bootstrapped like any dictionary.
2. Crash-safe DDL = transactional DD rows + the **InnoDB DDL log** for compensating physical operations. Recovery decides by transaction outcome — the same commit-point discipline as row data.
3. `I_S` became views; `.frm` files, and an entire genre of "orphaned table" bugs, are gone.
4. `ALTER` negotiates COPY / INPLACE / INSTANT with the engine — read `check_if_supported_inplace_alter` once and MySQL's online-DDL documentation becomes self-evident.

**Try it:** `SELECT * FROM information_schema.innodb_tables WHERE name LIKE 'mysql/%';` shows the DD tables living inside InnoDB; `SHOW CREATE VIEW information_schema.tables\G` reveals the `I_S`-as-view mechanism.

---

**Previous:** [Chapter 9 — Replication](#) · **Next:** [Chapter 11 — What Percona Adds](#)

# Chapter 11 — What Percona Adds

---

The delta between Percona Server 8.0 and upstream MySQL 8.0 — and what it teaches about the architecture's extension points. Source: `sql/threadpool_*.cc`, `storage/rocksdb/`, `plugin/`, `components/`, `sql/userstat.cc`

Percona Server is upstream MySQL plus patches that mostly slot into the extension points this series has mapped: an alternative *connection scheduler*, an alternative *storage engine*, extra *plugins/components*, and deeper *observability*. Reading the delta is a tour of where the architecture bends.

## 11.1 Thread pool (a Connection\_handler)

Upstream reserves the thread pool for MySQL Enterprise; Percona ships one in the tree (`sql/threadpool_unix.cc`, activated as the `thread_pool` plugin). It plugs into the exact seam from Chapter 1 — `Connection_handler_manager::load_connection_handler()` — replacing thread-per-connection with **thread groups**: connections are sharded across groups; each group has an epoll-based listener and a small elastic set of workers (`worker_main`, `threadpool_unix.cc:194`); a timer thread detects stalls and spawns relief workers. Result: ~constant threads under thousands of connections, no scheduler collapse under connection storms. Knobs: `thread_pool_size`, `thread_pool_oversubscribe`, `thread_pool_idle_timeout`.

## 11.2 MyRocks (a handler)

`storage/rocksdb/` (`ha_rocksdb.cc`, ~17.6k lines) wraps Facebook's RocksDB — an **LSM-tree** engine — behind the Chapter 6 interface. Where InnoDB updates B+tree pages in place (with redo/undo), an LSM engine turns every write into an append: memtable → sorted SST files → background compaction. The translation layer is instructive:

- `Rdb_key_def` (`rdb_datadic.h:270`) encodes index entries as **memcomparable keys** (byte order = logical order) so RocksDB's plain byte-sorted iterators implement SQL index scans — the packing/collation logic InnoDB does per-page comparison, done once at encode time.
- The PK is clustered (like InnoDB); secondary keys map to separate **column families**; engine metadata lives in a RocksDB system column family (`Rdb_dict_manager`) — the "dictionary stored in itself" pattern for the third time in these series.
- Trade-off vs InnoDB: much lower write amplification and better compression (great for write-heavy, SSD-bound, replica fleets) versus costlier reads/range scans and a weaker fit for mixed workloads — which is why it ships as an *option*, selectable per table with `ENGINE=ROCKSDB`.

(TokuDB, Percona's earlier alternative engine, is still present at `storage/tokudb/` but its build is hard-disabled — `CMakeLists.txt` opens with `RETURN()`. Engines come and go; the handler API abides.)

### 11.3 Security & operations plugins/components

The plugin API (same mechanism that registers engines, Ch. 6) carries most of the rest:

| feature            | location                                                                  | one-liner                                                                                 |
|--------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Audit log          | <code>plugin/audit_log/</code> ,<br><code>plugin/audit_log_filter/</code> | file/syslog audit trail of connections & statements                                       |
| PAM authentication | <code>plugin/percona-pam-for-mysql/</code>                                | OS-level auth (LDAP/2FA via PAM stack)                                                    |
| Keyring for Vault  | <code>plugin/keyring_vault/</code> ,<br><code>components/keyrings/</code> | encryption keys in HashiCorp Vault (TDE key management)                                   |
| Data masking       | <code>components/masking_functions/</code>                                | masking/anonymization UDFs                                                                |
| PROCFS             | <code>plugin/procfs/</code>                                               | host <code>/proc</code> metrics via <code>I_S</code> — observability without shell access |
| Backup locks       | <code>MDL_BACKUP_TABLES/BACKUP_LOCK</code> namespaces (Ch. 7)             | <code>LOCK TABLES FOR BACKUP</code> — the light-weight FTWRL replacement XtraBackup uses  |
| Column compression | DD system views<br><code>compression_dictionary*</code> (Ch. 10)          | per-column compression with shared dictionaries                                           |

### 11.4 Observability patches

The oldest and most-loved Percona deltas are diagnostic:

- **Extended slow log** (`sql/log.cc`, `log_slow_verbosity`): per-query InnoDB stats — pages read, I/O wait, lock wait — in the slow log; pre-dates `performance_schema` and still often more convenient. `pt-query-digest` grew up on this format.
- **User/table/index statistics** (`sql/userstat.cc`): `I_S` tables (`TABLE_STATISTICS`, `INDEX_STATISTICS`, `USER_STATISTICS`) counting rows read/changed per object — the quick answer to "which index is dead weight?"

### 11.5 What to remember

1. Percona's delta maps onto the architecture's four sanctioned extension points: connection handler, storage engine, plugin/component APIs, and instrumentation. Almost nothing forks the core query path — that's what keeps the fork maintainable.
2. MyRocks vs InnoDB is the cleanest real-world case study of the handler abstraction: LSM vs B+tree behind one interface, with memcomparable key encoding as the bridge.

3. The thread pool solves the C10K problem Chapter 1's model can't; backup locks solve the FTWRL problem Chapter 7 explained.
4. When evaluating "Percona vs upstream," you're mostly evaluating these tables — the SQL core is the same code you studied in Chapters 1-10.

**Try it:** `SHOW PLUGINS` on a Percona server, then match each row to a directory in `plugin/` or `storage/`; `SET GLOBAL log_slow_verbosity='full'` and read one slow-log entry.

---

**Previous:** [Chapter 10 — The Data Dictionary](#) · **Next:** [Chapter 12 — Capstone](#)

# Chapter 12 — Capstone: One Statement, Every Layer

The final exam for both series: narrate an UPDATE from TCP packet to replica, touching every mechanism studied.

## 12.1 The statement

```
UPDATE accounts SET balance = balance - 100 WHERE id = 42;
```

Autocommit on, binlog on (ROW), one replica attached, InnoDB table with PK `id`.

## 12.2 The full trace

### On the source:

1. **Wire** (Server Ch. 1): the connection's thread returns from `do_command`'s blocking read with a `COM_QUERY` packet → `dispatch_command` → `dispatch_sql_command`.
2. **Parse** (Ch. 2): lexer + Bison build `PT_*` nodes; contextualization fills a `Query_block` (one table, one assignment, WHERE `id=42`); digest recorded; `lex->sql_command = SQLCOM_UPDATE`.
3. **Resolve** (Ch. 3): MDL `SHARED_WRITE` on `accounts` + IX on GLOBAL/SCHEMA/COMMIT (Ch. 7); `TABLE_SHARE` from cache, `TABLE` instance bound, `ha_innobase` handler ready (Ch. 6); `fix_fields` binds `id` and `balance`.
4. **Optimize** (Ch. 4): range analysis on the PK finds `id=42` → single-row `EQ_REF`-style plan; AccessPath tree: `UPDATE_ROWS ← FILTER? ← INDEX` lookup.
5. **Execute** (Ch. 5): the update iterator pulls one row — `ha_index_read_map` (Ch. 6) → `row_search_mvcc`.
6. **Into InnoDB** (InnoDB series): B+tree descent to the leaf (Ch. 6 there), buffer pool fetch if needed (Ch. 3), row lock — `LOCK_X | LOCK_REC_NOT_GAP` on the PK entry (Ch. 8); MVCC not needed for a locking read. Row copied to `record[0]` via the row template.
7. The server computes the new row into `record[0]` (old in `record[1]`), calls `ha_update_row` → `row_update_for_mysql`: **undo record** written with the old balance (Ch. 7 there), row updated **in place** on the page inside a **mini-transaction** that emits **redo** (Ch. 4-5 there), page marked dirty in the flush list (Ch. 3). The `ha_*` wrapper appends a `Rows_log_event` update image to the THD's **binlog cache** (Ch. 8).
8. **Commit — the 2PC** (Ch. 7-8): `ha_commit_trans` → InnoDB **prepare** (`TRX_PREPARED`, redo flushed) → binlog group commit: FLUSH (GTID assigned, cache written), SYNC (one fsync covers the group; semi-sync waits for a replica ack here), COMMIT (InnoDB flips the undo state, releases the row lock — Ch. 7 there). OK packet to the client.

9. **Sometime later** (both series' background chapters): the page cleaner writes the dirty page through the doublewrite buffer; the checkpoint advances; purge reclaims the undo record once no read view needs the old balance.

**On the replica** (Ch. 9): the dump thread ships GTID + TABLE\_MAP + UPDATE\_ROWS + XID; the I/O thread appends to the relay log; a worker (scheduled by `last_committed` — safe because the transaction group-committed with its neighbors) applies it as a raw `ha_update_row`, committing the new position atomically with the data.

**If anything crashes anywhere:** InnoDB redo replays committed page changes and rolls back unprepared transactions (InnoDB Ch. 5); prepared-but-undecided transactions are resolved by the binlog XID scan (Ch. 7); the replica resumes from its transactional position (Ch. 9). No step in this trace can be half-done after recovery.

## 12.3 The five ideas you now own

Across ~25 chapters and two codebases, the same handful of ideas did all the work:

1. **Write-ahead + commit points.** Redo before pages, binlog XID before engine commit, DDL log before file ops, undo state flip as *the* commit — every durability story is "log the intention, then one atomic switch."
2. **MVCC: versions + visibility.** Undo chains and read views made readers lock-free in 2005; the same undo powers rollback, purge, and even INSTANT DDL's row versioning.
3. **Two-level concurrency.** Short physical latches (pages, mtr) under long logical locks (rows, metadata) — each level with its own deadlock strategy (ordering vs detection).
4. **Amortize the fsync.** Group commit in the embedded redo log, reinvented as the binlog stage pipeline, then reused as replication's parallelism oracle. One idea, three payoffs.
5. **Interfaces make evolution possible.** The handler API let B+trees and LSM trees coexist; the AccessPath tree let two optimizers coexist; the dictionary-in-itself pattern recurs in InnoDB, the DD, MyRocks, and replication positions.

## 12.4 Where to go next

- **Read code with a debugger, not a browser** — one breakpoint per chapter of this trace and a single UPDATE will show you everything in an afternoon.
- **Modern frontiers**, each an extension of a chapter here: the hypergraph optimizer (Ch. 4), Group Replication/Paxos (Ch. 9), redo-log and buffer-pool scalability work in 8.x InnoDB (InnoDB Ch. 3-5), HeatWave's secondary-engine hook (`override_executor_func`, Ch. 5's execution loop).
- **The history you now recognize:** when you read a MySQL changelog item, you can usually name the file it touches.

*Fim do curso — you can now narrate a database from the socket to the sector.*

---

**Previous:** [Chapter 11 — What Percona Adds](#) · **Series index:** [README](#) · **Companion:** [InnoDB Architecture Deep-Dive](#)